

Primeira ação

Antes de alterar qualquer configuração ou reiniciar serviços, minha primeira ação seria confirmar o incidente e coletar evidências.

A diferença entre 1.587 vendas aprovadas no gateway e apenas 421 registros em **lead_events** indica uma perda potencial de aproximadamente 1.166 leads. Como existe impacto financeiro e operacional (Call Center sem leads há horas), o objetivo inicial é identificar em qual etapa do fluxo ocorreu a interrupção.

Também comunicaria ao PO que o incidente está sendo investigado e que uma análise inicial será realizada antes de qualquer intervenção em produção.

Todas as hipóteses, primeiros dados / consultas, verificação, diferenciação de cenários e medidas preventivas foram pensadas a partir do que foi desenvolvido no teste técnico de esteira de processamento de webhooks.

Hipóteses Iniciais (ordenadas por probabilidade)

1. Consumer parado ou travado

Maior probabilidade porque existe uma grande diferença entre o volume recebido e o volume persistido.

Se os webhooks continuaram chegando normalmente, mas o consumer parou de consumir a fila, os eventos permaneceriam acumulados no RabbitMQ.

2. Falha de publicação para RabbitMQ

Os webhooks podem estar chegando corretamente, mas uma falha de conexão ou publicação pode impedir que os eventos sejam enviados para a fila principal.

3. Falha de decrypt ou validação de schema

Caso o gateway tenha alterado algum formato ou chave de criptografia, uma grande quantidade de eventos pode ter sido desviada para DLQ antes de entrar na esteira principal.

4. Problema de infraestrutura no RabbitMQ

Filas bloqueadas, consumidores desconectados ou crescimento anormal de mensagens pendentes podem impedir o processamento.

5. Gateway não enviou os eventos

Menos provável porque o dashboard do gateway informa 1.587 transações aprovadas, mas ainda deve ser validado através dos logs e registros recebidos.

Primeiros Dados e Consultas

Quantidade de webhooks recebidos

```
SELECT COUNT(*)  
FROM raw_payloads  
WHERE created_at >= NOW() - INTERVAL 4 HOUR;
```

O objetivo é confirmar se os webhooks chegaram à aplicação.

Quantidade persistida em lead_events

```
SELECT COUNT(*)  
FROM lead_events  
WHERE event = 'order.approved'  
AND persisted_at >= NOW() - INTERVAL 4 HOUR;
```

O objetivo é comparar entrada x persistência.

Erros de processamento

```
SELECT  
    source,  
    COUNT(*) total  
FROM lead_dead_letter  
WHERE created_at >= NOW() - INTERVAL 4 HOUR  
GROUP BY source;
```

O objetivo é identificar aumento anormal de falhas.

Último webhook processado com sucesso

```
SELECT  
    order_id,  
    gateway,  
    event,  
    persisted_at  
FROM lead_events  
ORDER BY persisted_at DESC  
LIMIT 1;
```

O objetivo é descobrir o último ponto saudável da esteira.

Primeiro erro após o último sucesso

```
SELECT
    source,
    error_message,
    created_at
FROM lead_dead_letter
WHERE created_at >
(
    SELECT MAX(persisted_at)
    FROM lead_events
)
ORDER BY created_at ASC
LIMIT 1;
```

O objetivo é identificar o primeiro evento de falha após a interrupção.

A partir da resposta dessa consulta, eu também analisaria o primeiro payload que iniciou a sequência de falhas. O objetivo seria rastrear o fluxo completo dessa mensagem, desde o recebimento no endpoint até sua persistência e eventual distribuição. Utilizando logs com **correlation_id**, registros de auditoria e tabelas intermediárias, seria possível acompanhar cada etapa da esteira e identificar exatamente onde ocorreu a interrupção. Essa abordagem reduz o tempo de investigação, pois permite reproduzir o caminho percorrido pelo primeiro evento afetado e encontrar a causa raiz do incidente antes que ele se propague para os demais registros.

Verificações RabbitMQ

Verificar filas acumuladas:

```
rabbitmqctl list_queues name messages consumers
```

Verificar consumidores ativos:

```
rabbitmqctl list_consumers
```

Caso exista acúmulo em **lead.received**, o problema provavelmente está no consumer.

Caso exista acúmulo em **dist.sms** o problema provavelmente está no distribuidor.

Como diferenciar os cenários

A) Gateway nunca enviou

- Poucos registros em **raw_payloads**
- RabbitMQ sem mensagens acumuladas
- Logs sem recebimento

B) Webhook chegou mas o decrypt falhou

- Registros em **raw_payloads**
- Crescimento da DLQ de decrypt
- Ausência de registros em **lead_events**

C) Consumer travou

- Muitos registros em **raw_payloads**
- Fila **lead.received** acumulando mensagens
- Poucos registros novos em **lead_events**

D) Distribuidor não consumiu

- Registros normalmente persistidos em **lead_events**
- Fila **dist.sms** acumulada
- Status permanecendo como **pending** em **distribution_status**

Plano de Reprocessamento

Primeiramente identificaria os eventos que chegaram mas não foram persistidos.

Exemplo:

```
SELECT rp.id
FROM raw_payloads rp
LEFT JOIN lead_events le
  ON rp.correlation_id = le.correlation_id
WHERE le.id IS NULL;
```

Os registros encontrados seriam republicados para a fila principal.

Como a aplicação utiliza idempotência baseada em **(transaction_id, event)**, os 421 registros já processados não seriam duplicados.

Estratégia:

1. Selecionar eventos faltantes.
2. Republicar para **lead.received**.
3. Permitir que a camada de idempotência descarte duplicados.
4. Monitorar consumo e persistência até eliminar o backlog.

Medidas Preventivas

1. Alertas de backlog

Criar alerta para filas com crescimento anormal:

lead.received
dist.sms

2. Métricas de funil

Monitorar:

raw_payloads
lead_events
distribution_status

Diferenças significativas entre essas etapas devem gerar alerta automático.

3. Alertas de DLQ

Monitorar crescimento das tabelas e filas de dead letter.

Qualquer aumento repentino de erros de decrypt, schema ou processamento deve gerar notificação imediata para o time.

Complemento

Esse último ponto foi pensado a partir da implementação de métricas com Prometheus no próprio teste técnico. As métricas foram expostas em etapas críticas da esteira, como recebimento de leads, erros de processamento, latência, lag entre gateway e persistência e distribuição SMS. Essa abordagem permite criar consultas e alertas diretamente sobre o comportamento da aplicação, sem depender apenas da análise de logs. Como evolução natural da solução, o Grafana poderia ser integrado consumindo essas métricas do Prometheus, fornecendo dashboards operacionais e alertas visuais para acompanhamento da saúde do sistema, identificação de gargalos e detecção antecipada de incidentes.